

```

Thu Jun 18 15:16:08 2009 /sbin/ip link set dev tun0 up mtu 1500
Thu Jun 18 15:16:08 2009 /sbin/ip addr add dev tun0 local 10.8.0.2 peer
10.8.0.1
Thu Jun 18 15:16:08 2009 Data Channel MTU parms { L:1545 D:1450 EF:45
EB:135 ET:0 EL:0 AF:3/1 }
Thu Jun 18 15:16:08 2009 Local Options hash (VER=V4): '48993abd'
Thu Jun 18 15:16:08 2009 Expected Remote Options hash (VER=V4):
'4b91e501'
Thu Jun 18 15:16:08 2009 Socket Buffers: R=[124928->131072] S=[124928-
>131072]
Thu Jun 18 15:16:08 2009 UDPv4 link local (bound): [undef]:1194
Thu Jun 18 15:16:08 2009 UDPv4 link remote: 173.45.227.64:1194
Thu Jun 18 15:16:14 2009 Peer Connection Initiated with
173.45.227.64:1194
Thu Jun 18 15:16:14 2009 Initialization Sequence Completed

```

Now you can *ping* the client from the server and server from the client:

```

root@vpn.unixclinic.net # ping -c2 10.8.0.2
PING 10.8.0.2 (10.8.0.2) 56(84) bytes of data:
64 bytes from 10.8.0.2: icmp_seq=1 ttl=64 time=256 ms
64 bytes from 10.8.0.2: icmp_seq=2 ttl=64 time=256 ms

```

```

root@client # ping -c2 10.8.0.1
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data:
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=256 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=255 ms

```

This completes our static key-based set-up. Let us now move on to certificate-based authentication.

Certificate-based authentication

The static key-based set-up we saw earlier limits us to one client connecting to one server and also does not provide enough security. OpenVPN's support for PKI (Public Key Infrastructure) allows multiple clients to connect to a server, securely. The number of clients is only limited by the available bandwidth and the hardware resources available with the server.

If you are setting up OpenVPN for your corporate use, then you may want to use a commercial CA for issuance of certificates. However, if you want to avoid that additional cost, you can create your own CA for your internal use.

For those who want to set up a quick PKI for use by OpenVPN only, or for laboratory testing of the concept before rolling out the production set-up, OpenVPN bundles a set of scripts and tools called *easy-rsa*. You will get *easy-rsa* from any OpenVPN version that is downloaded from the main website.

If you have installed the packaged *openvpn* from your distribution, you should typically find this at */usr/share/doc/openvpn/examples/easy-rsa* in Debian and */usr/share/openvpn/easy-rsa* in CentOS. Please feel free to copy this directory to a convenient location of your

choice. I usually put this in the */opt/openvpn-easy-rsa-2.0/* directory and place that in my path, so I will assume that is the case here:

```
root@vpn.unixclinic.net # mkdir -p /opt/easy-rsa/keys
```

If you have a CentOS server then copy as follows:

```
root@vpn.unixclinic.net # cp -r /usr/share/openvpn/easy-rsa/2.0/* /opt/easy-rsa/
```

On a Debian server, copy the *easy-rsa* as follows:

```
root@vpn.unixclinic.net # cp -r /usr/share/doc/openvpn/examples/easy-rsa/2.0/* /opt/easy-rsa/
```

Next, put the directory in your path to work more easily with *easy-rsa*.

```
root@vpn.unixclinic.net # export PATH=$PATH:/opt/easy-rsa
```

The file */opt/easy-rsa/vars* contains the required environment variables for generating the keys. This file is commented very well and hence I would not describe it here. Most of the variables can be left to their default values. Make sure you change the *EASY_RSA* variable to the directory in which you've copied the *easy-rsa* tools. The default value may not suit our requirements:

```
export EASY_RSA="/opt/easy-rsa"
```

You may want to change the following variables. Given below are the values for these variables that are used for the purpose of this article:

```

export KEY_COUNTRY="IN"
export KEY_PROVINCE="UA"
export KEY_CITY="Nainital"
export KEY_ORG="Unixclinic"
export KEY_EMAIL="casupport@unixclinic.net"

```



Note: By default, the expiry of the root CA key and certificates is set to 3650 days (10 years). Practically, the certificates should expire after one or two years. So I went ahead and changed the *KEY_EXPIRE* to 365 days.

Now let us start generating the necessary keys and certificates. To start with, we need to source the variables specified in the *vars* file and make sure that the existing keys directory is cleaned. Pay extra attention to the *'.'* in the following command. *'.'* will make sure that the environment variables will be set up in the current shell.